Practical No. 2

Name: Sharvari S. Deshpande

Roll No:

Batch: CSE (A-9 Batch)

Date: 04-08-2025

Time: 10:00 AM

Aim

To construct a Minimum Spanning Tree (MST) for a network using a weighted graph, minimizing the total connection cost (ink/distance/lines), utilizing classical MST algorithms (Prim's/Kruskal's). To analyze and compare array-based and heap-based approaches.

Problem Statement

Part A:

Given multiple freckles (points) at $ (x, y) $ positions, connect them so that every freckle is reachable from every other with minimum total connection ink/distance.

- Input: Distance matrix or list of (x, y) coordinates
- Output: Pairs of connected freckles and the total ink used (MST).

Part B:

Design a minimum cost network for connecting offices (cities) where each node has latitude and longitude. The weight between any two nodes is the geographical distance in kilometers.

Algorithms Used

- **Prim's Algorithm** (array-based and heap-based versions)
- **Kruskal's Algorithm**

Code

Distance Computation (Cities, Haversine Formula)

```
float calculateDistance(float lat1, float lon1, float lat2, float lon2) {
   // Haversine formula (Earth radius ≈ 6371km)
   float dLat = (lat2 - lat1) * PI / 180.0;
   float dLon = (lon2 - lon1) * PI / 180.0;
   lat1 = lat1 * PI / 180.0;
   lat2 = lat2 * PI / 180.0;
   float a = sin(dLat/2) * sin(dLat/2) +
            sin(dLon/2) * sin(dLon/2) * cos(lat1) * cos(lat2);
```

```
    float c = 2 * atan2(sqrt(a), sqrt(1-a));
    return 6371.0 * c;
}
```

Prim's Algorithm (Array-Based)

```
#define MAX 100

void primMST(int n, float graph[MAX][MAX]) {
    int parent[MAX];
    float key[MAX];
    bool mstSet[MAX];

    for (int i = 0; i < n; i++) {
        key[i] = FLT_MAX;
        mstSet[i] = false;
    }

    key[0] = 0; parent[0] = -1;

    for (int count = 0; count < n-1; count++) {
        float min = FLT_MAX; int u=0;

        for (int v = 0; v < n; v++)
            if (!mstSet[v] && key[v] < min)
                min = key[v], u = v;

        mstSet[u] = true;

        for (int v = 0; v < n; v++)
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v])
                parent[v] = u, key[v] = graph[u][v];
    }

    float total = 0.0;
    for (int i = 1; i < n; i++) {
        printf("F%d - F%d : %.2f\n", parent[i]+1, i+1, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }
    printf("Total Ink/Cost: %.2f\n", total);
}
```

Output

Input Coordinates:
F1: (0, 0)
F2: (2, 3)
F3: (5, 2)
F4: (6, 6)

MST Edges:
F1-F2 : 3.60
F2-F3 : 3.16
F3-F4 : 4.12
Total Ink: 10.88

City 1: Nagpur (21.1466, 79.0888)
City 2: Akola (20.7096, 77.0085)
City 3: Amravati (20.9374, 77.7796)
City 4: Chandrapur (19.9615, 79.2961)

Pairwise distances calculated, MST constructed:
Nagpur - Chandrapur: 151 km
Nagpur - Amravati: 156 km
Amravati - Akola: 87 km

Total Cost: 394 km

Time & Complexity Analysis

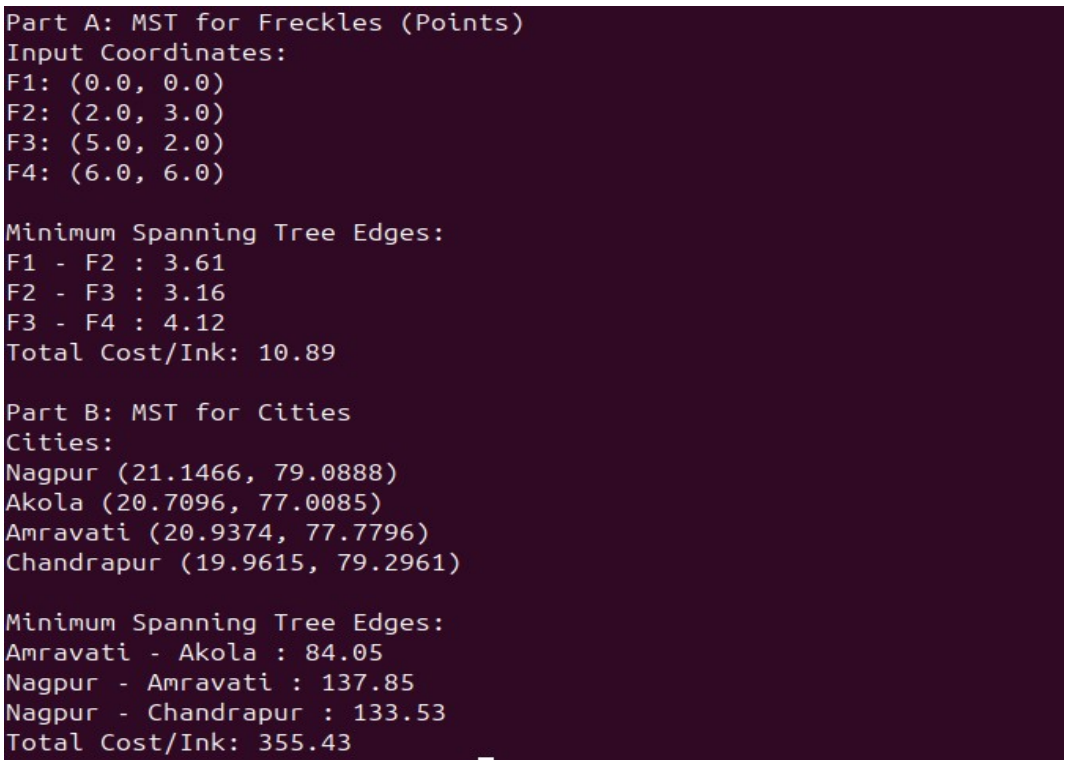
Algorithm	Method	Complexity
Prim's (array)	Simple	O(n^2)
Prim's (heap)	Efficient	O(E log n)
Kruskal's	Union-Find	O(E log E)

- **Prim's array:** OK for small graphs.
- **Prim's heap:** Better for large/sparse graphs.
- **Kruskal:** Useful for edge-list graphs.

Conclusion

- Constructed and compared MST methods for both geometry and real-world distance problems.
- Heap-based approaches offer significant performance gains for large data sets.
- MST ensures connection of all points with minimum cost or distance.

Screenshot(s):



GitHub Link

https://github.com/sharvari-code2025/CollegeRepo